Архитектура платформы корпоративных знаний

Назначение

Платформа помогает инженеру находить сведения в технических документах и связывать оборудование через общий контекст. Насос Н-17 упоминается с контуром К-2 в одном документе, а контур К-2 с датчиком Д-8 в другом. GraphRAG находит двухшаговую связь и возвращает оба источника. Документ с бюджетом Н-17 доступен только сотрудникам с соответствующей ролью.

Обработка выполняется локально. Система использует готовую модель, обучение не требуется. Демонстрационные документы синтетические. Найденное совместное упоминание требует интерпретации специалистом и не означает физическую или причинную зависимость.

Требования и ограничения

Основные требования: автономный inference, GraphRAG, нелинейный агент, права на уровне документов и производных чанков/рёбер, проверяемые цитаты, наблюдаемость. Запрос ограничен 2000 символами, документ 25 MiB и 100 страницами. Обход графа ограничен двумя шагами, 32 чанками и 128 сущностями; после ранжирования остаётся до шести чанков. Агент делает до двух попыток поиска, recursion limit равен 20.

Ориентир для пилотного сценария: p95 до 5 секунд при четырёх параллельных запросах. Измеренные RPS и задержки приведены в PERFORMANCE.md; они относятся к указанному синтетическому workload.

Представления архитектуры

В архитектура.drawio и архитектура.pdf представлены восемь схем: C1 Context, C2 Containers, C3 Agent Components, C4 Code, Deployment, Sequence, ER и Data Flow.

C1 показывает инженера, администратора, файловый архив и проектные интерфейсы ERP/CRM. Реализованная загрузка документов выполняется через CLI.

C2 разделяет Control Plane и Data Plane. Control Plane включает frontend, Nginx gateway, FastAPI и LangGraph. Data Plane содержит каноническое хранилище SQLite, производные индексы Qdrant/Neo4j, OCR и GPU-serving. API, агент и ingestion объединены в backend-образ; роли разделены в коде.

C3 раскрывает guard, память, planner, retriever, graph tool, reranker, generator и validator. C4 связывает их с Principal, State, Agent, Store, Generator и Indexes.

Загрузка и модель данных

Администратор задаёт tenant и роли документа. Text/Markdown читаются как UTF-8. PDF разбирается с привязкой к страницам; страницы без текстового слоя обрабатываются Tesseract rus+eng. Отсутствие OCR или распознаваемого текста приводит к ошибке загрузки. Распознавание геометрии чертежей и структуры сложных таблиц не реализовано.

Sanitizer маскирует email и телефоны, известные шаблоны подозрительных инструкций блокируют загрузку. Документ, версия и hash сохраняются транзакционно. Текст разбивается на чанки с перекрытием. Идентификаторы оборудования образуют Entity; совместные упоминания в чанке создают CO_MENTION с provenance исходного фрагмента.

SQLite хранит Document, Chunk, Entity, Mention, Edge, grants, историю сессий и метаданные индексов. Qdrant содержит векторы и ACL payload без полного текста. Neo4j содержит производный граф для визуализации и проверки связей. Канонические права всегда читаются из SQL.

Обработка запроса

1. Guard проверяет ввод и маскирует персональные данные.

2. Planner проверяет владельца сессии и разрешённую память.

3. Retriever ищет среди разрешённых чанков.

4. Graph Tool расширяет найденные связи с проверкой provenance и ACL.

5. Reranker выбирает ограниченный контекст.

6. Generator вызывает локальную модель для выбора source IDs.

7. Сервер восстанавливает точные фрагменты и offsets, validator проверяет доказательства.

8. Store повторно проверяет tenant, grants, версию текста и владельца сессии в транзакции BEGIN IMMEDIATE, сохраняет историю и формирует результат.

При недостатке evidence условный переход возвращает управление к planner. После второй неудачной попытки возвращается no_evidence. SSE отправляет один полностью проверенный результат. Отзыв права после завершения транзакции не отзывает уже переданные байты.

Для небольшого автономного корпуса применяется lexical hashing и ранжирование по перекрытию слов. Локальный SentenceTransformer подключается через EMBEDDING_MODEL_DIR с local_files_only. Смена модели или размерности требует перестроения индекса.

LLM и GPU

Qwen2.5-7B-Instruct-AWQ обслуживается vLLM 0.10.2, kernel AWQ Marlin, half, prefix caching. Контекст 4096 токенов, резерв ответа 700 и запас 64, max-num-seqs 4, max-num-batched-tokens 1024, gpu-memory-utilization 0,80. Веса и tokenizer предварительно доставляются локально; модель распространяется по Apache-2.0.

TokenBudget использует фактический tokenizer и chat template сервера. Контекст упаковывается полными текстовыми единицами. При нехватке контекста или неверном выборе модели используется явный extractive fallback с повторной валидацией. Ответ не содержит свободного пересказа непроверенных фактов. Без LLM_URL extractive-режим работает без модели.

Расчёт KV для Qwen2.5: 28 слоёв × 4 KV-heads × 128 head dimension × 2 K/V × 2 bytes = 57 344 bytes/token. Для 4 × 4096 токенов требуется около 0,875 GiB KV без прочего overhead. Профиль рассчитан на RTX 4070 12 GiB. Offloading возможен при меньшем объёме VRAM, но увеличивает задержку и требует отдельных измерений.

Защита данных

Bearer token сопоставляется с серверным provisioning-файлом. Tenant и роли не принимаются из пользовательского JSON. Сессия принадлежит конкретному пользователю. Grants наследуются документом, чанками и рёбрами; общая Entity не объединяет права разных документов. ACL применяется до поиска и обхода, а затем повторно перед ответом.

SQL параметризован; инструменты не выполняют пользовательские shell-команды, SQL или произвольные URL. Клиент модели допускает только INTERNAL_HOSTS. История хранит маскированный вопрос и source IDs, источники при чтении заново авторизуются. Журнал событий содержит минимальные метаданные без цитат; цепочка hashes позволяет проверять целостность при

доверенном anchor. Regex-фильтры являются ограниченным слоем защиты и требуют дополнения доменными проверками при расширении корпуса.

Развёртывание

Локальный Docker Compose использует internal network, loopback-порты, non-root/read-only backend и отдельную edge-сеть gateway. DB-порты не публикуются в DMZ. Образы, зависимости и веса готовятся до переноса в закрытый контур; runtime не обращается к внешним inference API.

Целевая схема Deployment выделяет DMZ с TLS reverse proxy, Internal Control с агентами и ingestion, Internal Data с БД и GPU. Между зонами предусмотрены allowlist, mTLS и service identities; Vault хранит секреты. L7 proxy распределяет запросы между готовыми API-репликами, serving router учитывает readiness и очередь GPU. Эти компоненты целевой схемы расширяют локальный MVP. Для нескольких API-реплик нужны общая PostgreSQL и общий checkpointer вместо отдельных SQLite.

Наблюдаемость и эксплуатация

OpenTelemetry передаёт spans query, agent.guard/plan/retrieve/expand/rerank/generate/validate и llm.generate в Jaeger. Тексты вопросов, ответов и источников не включаются в attributes. Prometheus собирает HTTP status, latency, фактические input/output tokens и состояние зависимостей; Grafana отображает метрики. vLLM публикует собственные serving-метрики.

/health отражает доступность API, /ready проверяет необходимые SQL, identity, index и model dependencies. Индексы создаются как новые версии, проверяются и публикуются одним активным указателем SQL. Старые версии сохраняются для отката. Резервное копирование использует согласованный SQL snapshot; производные индексы восстанавливаются из канонических данных. Порядок действий описан в RUNBOOK.md.

Проверка

Unit- и интеграционные тесты покрывают граф, разграничение доступа, историю, отзыв прав во время генерации, PII, подозрительные документы, ложные цитаты, offsets, token budget, индексы и HTTP/SSE. Команды приведены в TESTING.md. Нагрузочные измерения описаны отдельно в PERFORMANCE.md.